

AUTOMATE SINCRONE



AUTOMATE SINCRONE

Implementare cu memorii

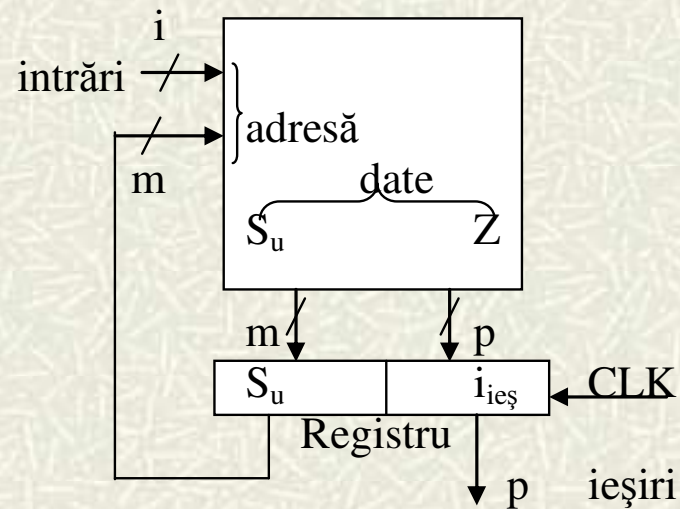
- Memorii: ROM, RAM
- Memoriile ROM:
 - Intrări = adrese – n biți
 - ieșiri = date
 - Număr locații adresate – 2^n
 - Lungime cuvânt memorie – m
 - Capacitate memorie – $C = 2^n \times m$
 - Acces la date:
 - fixare adrese; generare CS; după trecerea timpului de acces la memorie datele apar pe liniile de date

METODE DE IMPLEMENTARE

- 1. Adresare pe arc
- 2. Adresare pe stare
 - a. cu adresă pereche
 - b. cu adresă presupusă
 - c. cu format variabil

1. ADRESARE PE ARC

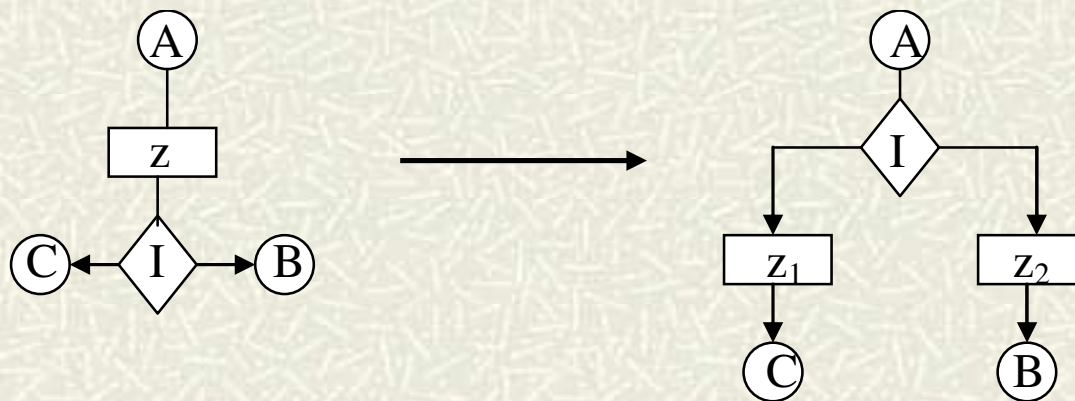
Schema bloc



- i = nr. intrări
- m = nr. de biți pt. codificarea stării următoare S_u
- p = nr. de ieșiri distincte
- $n = i + m$ = nr. biți de adresă
- $l = m + p$ = lungimea cuvântului de memorie
- $C = 2^n \times l$ = capacitatea memoriei
- Structură cuvânt de memorie
 - m biți pentru codificarea stării următoare S_u
 - p biți pentru ieșiri $i_{ieș}$

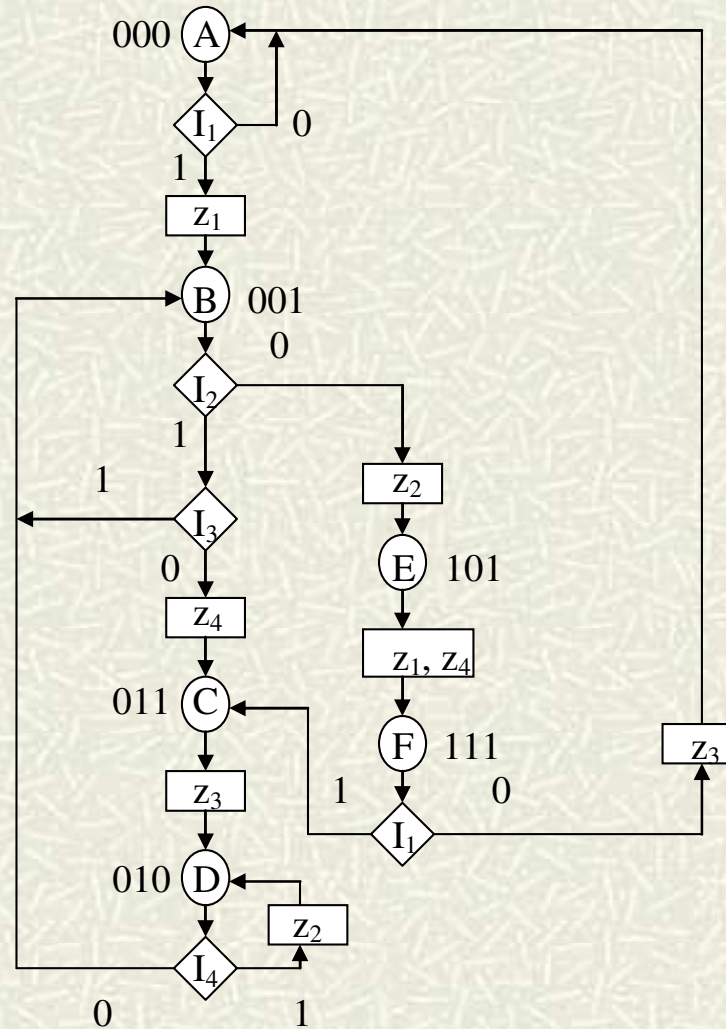
1. ADRESARE PE ARC

- Valabilă pentru **automate de tip Mealy**
 - starea următoare determinată de intrări și de starea prezentă
- Pentru alte tipuri de automate
 - în organigramă - translatarea ieșirilor la capetele arcelor:



1. ADRESARE PE ARC

Exemplu



1. ADRESARE PE ARC

Exemplu

- Necesarul de memorie:
 - $i = 4$ intrări
 - $m = 3$ biți pt. codificarea stării următoare
 - $p = 4$ ieșiri
 - $n = i + m = 7$ biți de adrese
 - $l = m + p = 7$ biți lungimea cuvântului de memorie
 - $C = 2^n \times l = 896$ biți – capacitatea memoriei
- **Observație**
 - nu este necesară codificarea adiacentă a stărilor (nu avem o implementare cu bistabile!)

1. ADRESARE PE ARC

Harta memoriei

- Adresele și informația care va fi înscrisă în memorie

	Stări	Adrese							Date							
		Q ₂	Q ₁	Q ₀	I ₄	I ₃	I ₂	I ₁	Q ₂ ^t	Q ₁ ^t	Q ₀ ^t	z ₄	z ₃	z ₂	z ₁	
00	A	0	0	0	x	x	x	0	0	0	0	0	0	0	0	00
01		0	0	0	x	x	x	1	0	0	1	0	0	0	1	11
10	B	0	0	1	x	x	0	x	1	0	1	0	0	1	0	52
12		0	0	1	x	0	1	x	0	1	1	1	0	0	0	38
16		0	0	1	x	1	1	x	0	0	1	0	0	0	0	10
30	C	0	1	1	x	x	x	x	0	1	0	0	1	0	0	24
20	D	0	1	0	0	x	x	x	0	0	1	0	0	0	0	10
28		0	1	0	1	x	x	x	0	1	0	0	0	1	0	22
50	E	1	0	1	x	x	x	x	1	1	1	1	0	0	1	79
70	F	1	1	1	x	x	x	0	0	0	0	0	1	0	0	04
71		1	1	1	x	x	x	1	0	1	1	0	0	0	0	30

Starea actuală

Intrări

Starea următoare

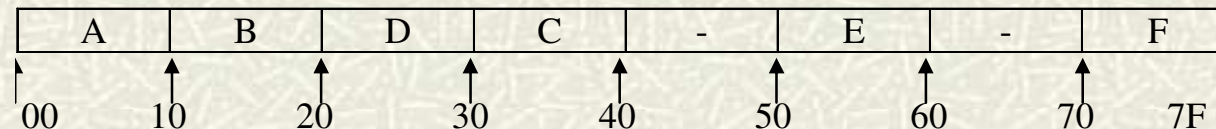
Ieșiri

1. ADRESARE PE ARC

Harta memoriei

- Dispunerea locațiilor pentru stări

Adresă (hexa)	Conținut (hexa)	Stare
00 – 0E (valori pare)	00	A
01 – 0F (valori impare)	11	A
10 – 11; 14 – 15; 18 – 19; 1C – 1D	52	B
12 – 13; 1A – 1B	38	B
16 – 17; 1E – 1F	10	B
20 – 27	10	D
28 – 2F	22	D
30 – 3F	24	C
40 – 4F	-	-
50 – 5F	79	E
60 – 6F	-	-
70 – 7E (valori pare)	04	F
71 – 7F (valori impare)	30	F

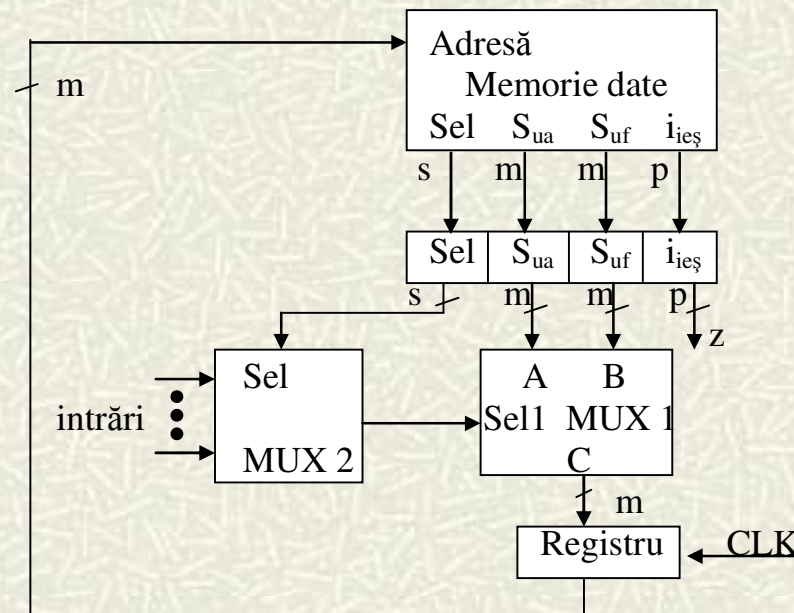


- Memoria este ineficient ocupată

2. ADRESARE PE STARE

a. Adresare pe stare cu adresă pereche

Schema bloc

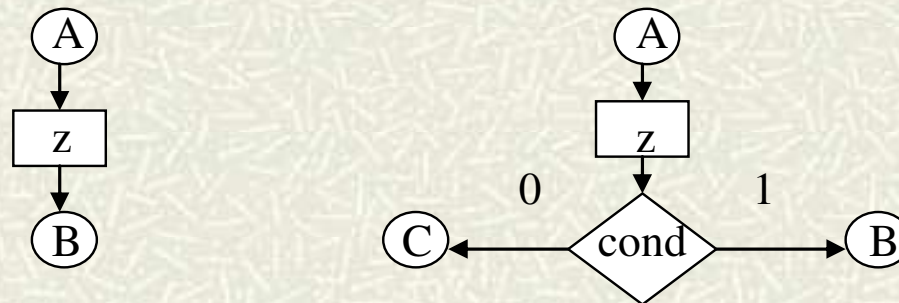


- Sel = selecție – s biți
- S_{ua} = starea următoare pentru intrări adevărate – m biți
- S_{uf} = starea următoare pentru intrări false – m biți
- $i_{ieș}$ = ieșiri – p biți
- $C = 2^m \times (s + 2m + p)$ = capacitatea memoriei

2. ADRESARE PE STARE

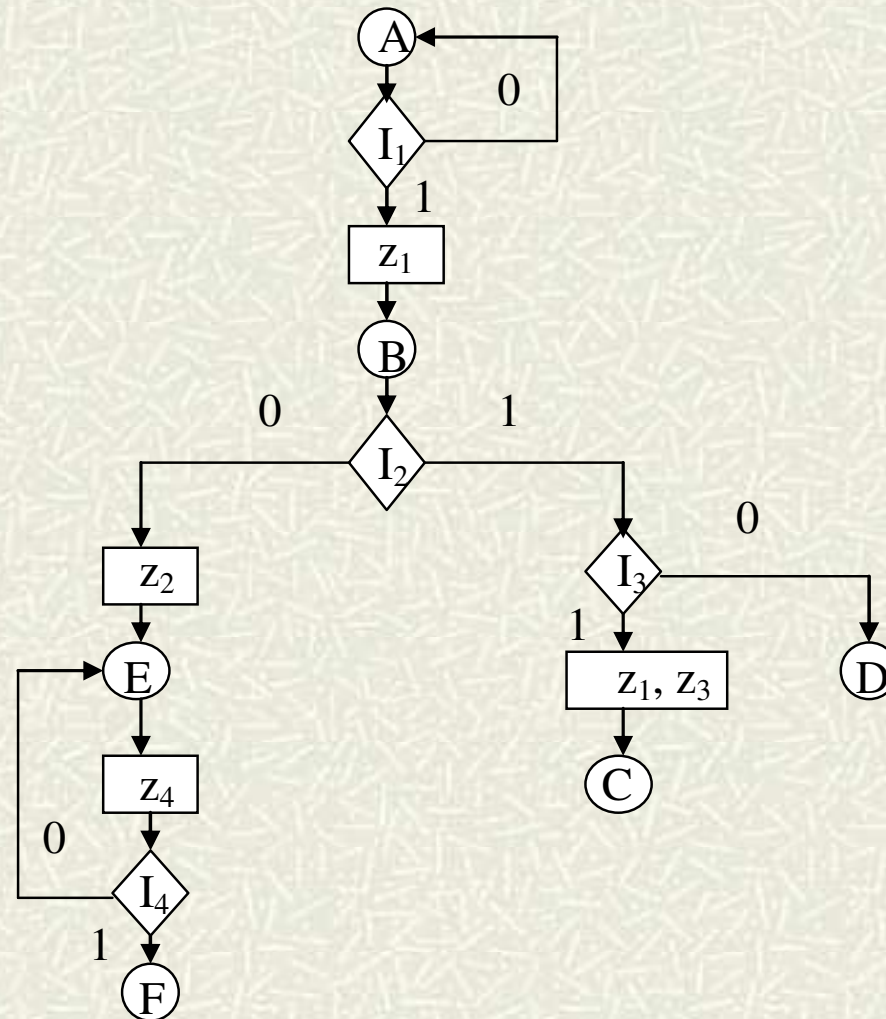
a. Adresare pe stare cu adresă pereche

- Valabilă pentru automate de tip Moore
 - ieșirile depind doar de variabilele de stare
- Pentru alte tipuri de automate
 - în organigramă – modificări pentru a obține structuri doar de forme:



2. ADRESARE PE STARE

Exemplu

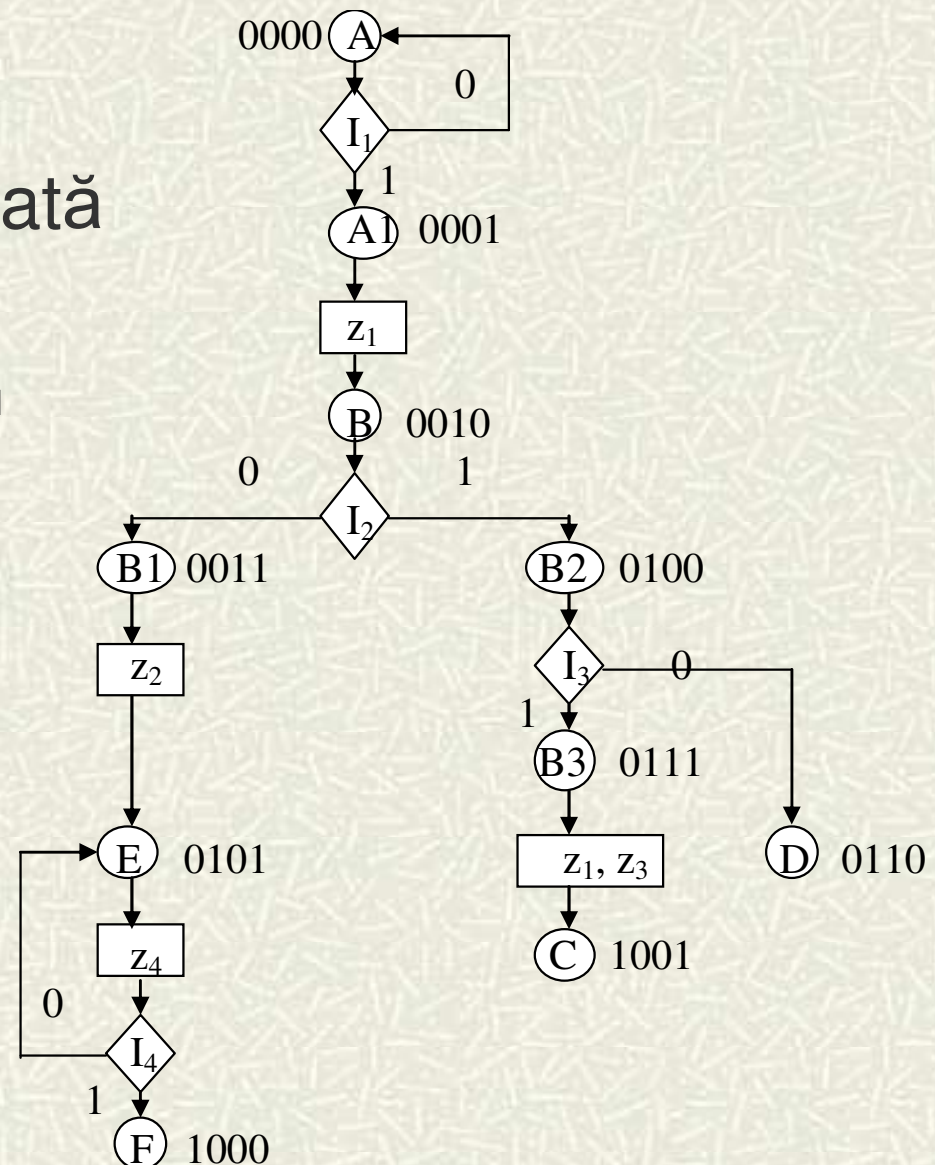


2. ADRESARE PE STARE

Exemplu

■ Organigrama modificată

- după A, pentru $I_1=1$
 - se introduce starea A_1
- după B, pentru $I_2=0$
 - se introduce B_1
- după B, pentru $I_2=1$
 - se introduce B_2
- după B_2 , pentru $I_3=1$
 - se introduce B_3



2. ADRESARE PE STARE

Exemplu

- Codificarea (arbitrară) a stărilor și condițiile de test pentru intrări

				S_1S_0	
A	0000	0	I_1	00	0
A1	0001	1	I_2	01	1
B	0010	2	I_3	10	2
B1	0011	3	I_4	11	3
B2	0100	4			
D	0110	6			
B3	0111	7			
C	1001	9			
E	0101	5			
F	1000	8			

2. ADRESARE PE STARE

a. Adresare pe stare cu adresă pereche

- Capacitatea memoriei: $C = 2^m \times (s+2m+p) = 224$ biți
- Harta memoriei

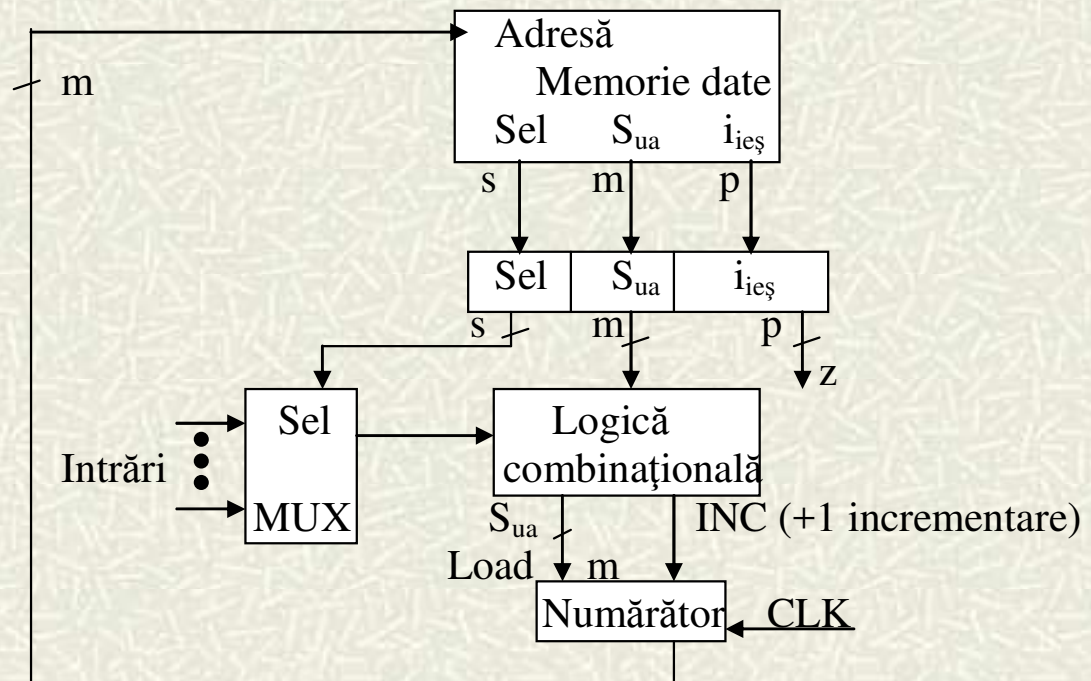
	Adresă stare prezentă	Sel (cond. de test)	S _{ua}	S _{uf}	Z_4, Z_3, Z_2, Z_1 i _{ieș}
A	0 (0000)	00 (I ₁)	1 (0001)	0 (0000)	0 (0000)
A1	1 (0001)	xx	2 (0010)	2 (0010)	1 (0001)
B	2 (0010)	01 (I ₂)	4 (0100)	3 (0011)	0 (0000)
B1	3 (0011)	xx	5 (0101)	5 (0101)	2 (0010)
B2	4 (0100)	10 (I ₃)	7 (0111)	6 (0110)	0 (0000)
E	5 (0101)	11 (I ₄)	8 (1000)	5 (0101)	8 (1000)
D	6 (0110)	În funcție de dezvoltarea ulterioară			
B3	7 (0111)	xx	9 (1001)	9 (1001)	5 (0101)
F	8 (1000)	În funcție de dezvoltarea ulterioară			
C	9 (1001)	În funcție de dezvoltarea ulterioară			

- Număr mai mic de locații de memorie, dar număr mai mare de biți / locație

2. ADRESARE PE STARE

b. Adresare pe stare cu adresă presupusă

Schema bloc

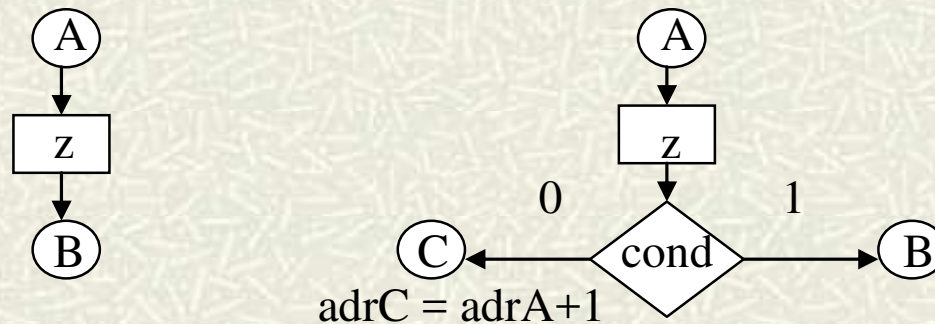


- Sel = selecție – s biți
- S_{ua} = starea următoare pentru intrări adevărate – m biți
- i_{ieș} = ieșiri – p biți
- $C = 2^m \times (s+m+p)$ = capacitatea memoriei

2. ADRESARE PE STARE

b. Adresare pe stare cu adresă presupusă

- Structurile admise în organigramă au o **condiție în plus**:
 - **codul pentru $S_{uf} = \text{codul pentru } S_{actuală} + 1$**



2. ADRESARE PE STARE

Exemplu

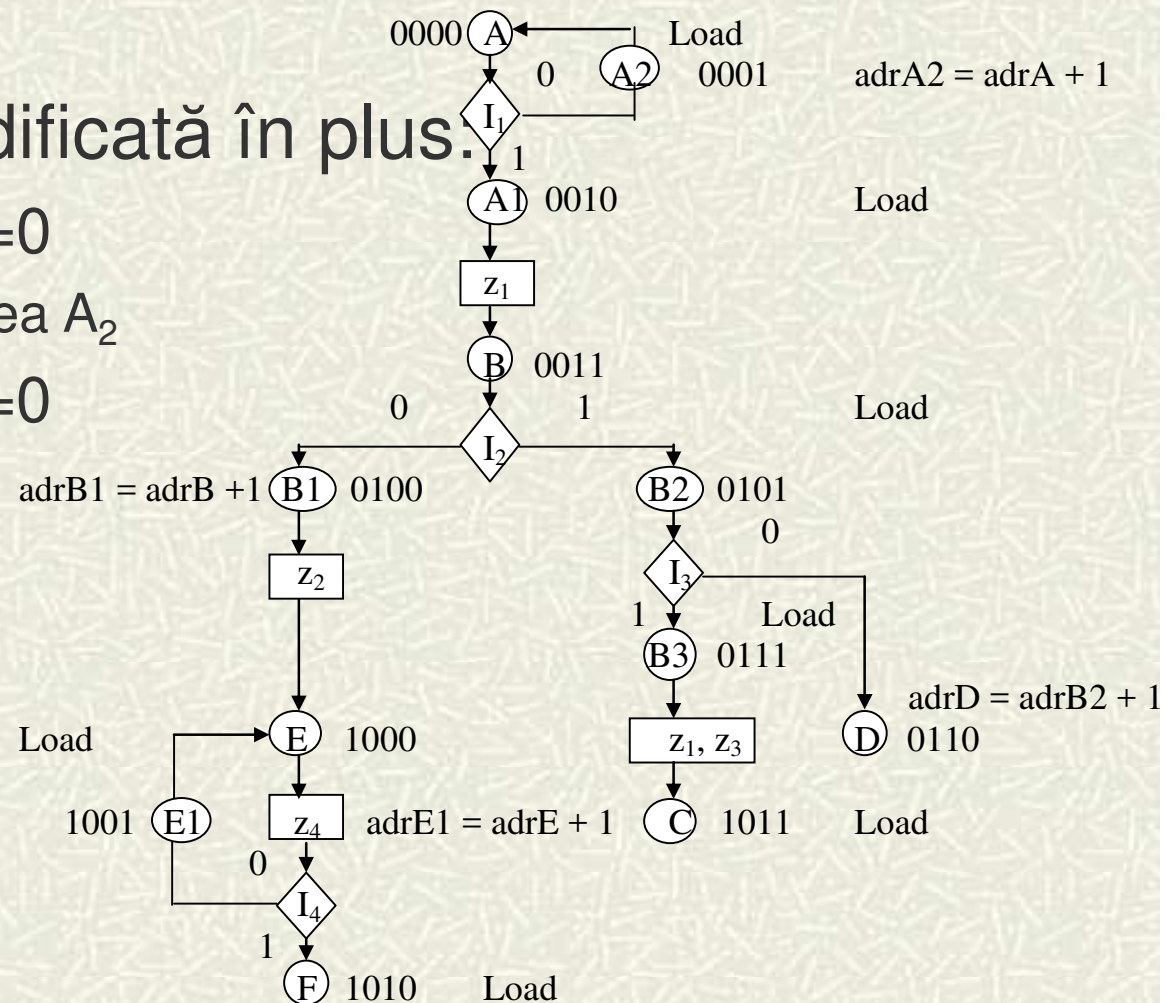
■ Organigrama modificată în plus:

■ după A, pentru $I_1=0$

- se introduce starea A_2

■ după E, pentru $I_4=0$

- se introduce E_1



2. ADRESARE PE STARE

b. Adresare pe stare cu adresă presupusă

- Capacitatea memoriei: $C = 2^m \times (s+m+p) = 160$ biți
- Harta memoriei

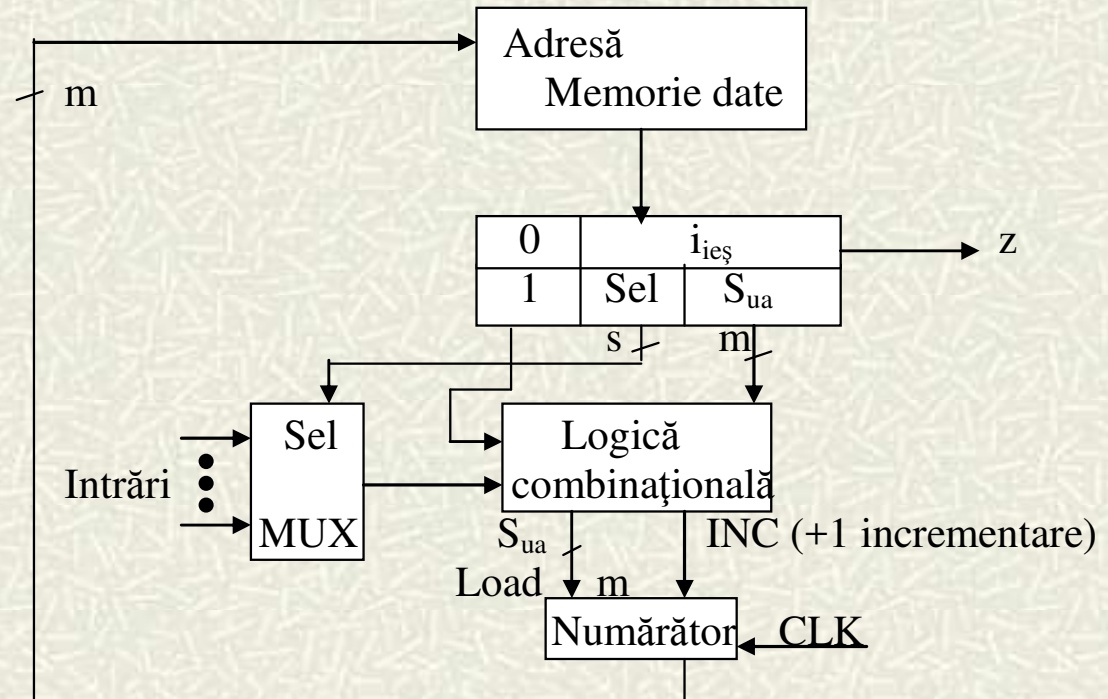
	Adresă stare prezentă	Sel (cond. de test)	S_{ua}	Z_4, Z_3, Z_2, Z_1 $i_{ieș}$
A	0 (0000)	00 (I_1)	2 (0010)	0 (0000)
A2	1 (0001)	xx	0 (0000)	0 (0000)
A1	2 (0010)	xx	3 (0011)	1 (0001)
B	3 (0011)	01 (I_2)	5 (0101)	0 (0000)
B1	4 (0100)	xx	8 (1000)	2 (0010)
B2	5 (0101)	10 (I_3)	7 (0111)	0 (0000)
D	6 (0110)	În funcție de dezvoltarea ulterioară		
B3	7 (0111)	xx	B (1011)	5 (0101)
E	8 (1000)	11 (I_4)	A (1010)	8 (1000)
E1	9 (1001)	xx	8 (1000)	0 (0000)
F	A (1010)	În funcție de dezvoltarea ulterioară		
C	B (1011)	În funcție de dezvoltarea ulterioară		

- Lipsește coloana pentru S_{uf}

2. ADRESARE PE STARE

c. Adresare pe stare cu format variabil

Schema bloc



- Sel = selecție – s biți
- S_{ua} = starea următoare pentru intrări adevărate – m biți
- $i_{ieș}$ = ieșiri
- $C = 2^m \times (s+m+1)$ = capacitatea memoriei

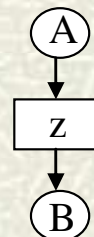
2. ADRESARE PE STARE

c. Adresare pe stare cu format variabil

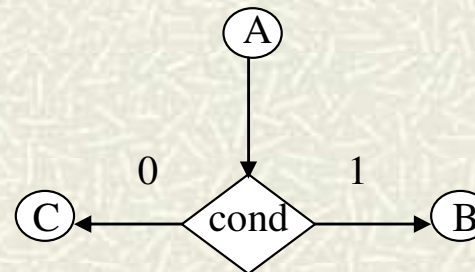
- Avem 2 tipuri de cuvinte de memorie

0	Ieșiri		format 0
1	Sel	S_{ua}	format 1

- Primul bit al cuvântului de memorie stabilește tipul
- Structurile admise în organigramă au **condițiile**:
 - format 0 – **codul pentru $S_u = \text{codul pentru } S_{\text{actuală}} + 1$**
 - format 1 – **codul pentru $S_{uf} = \text{codul pentru } S_{\text{actuală}} + 1$**



$\text{adrB} = \text{adrA} + 1$
format 0

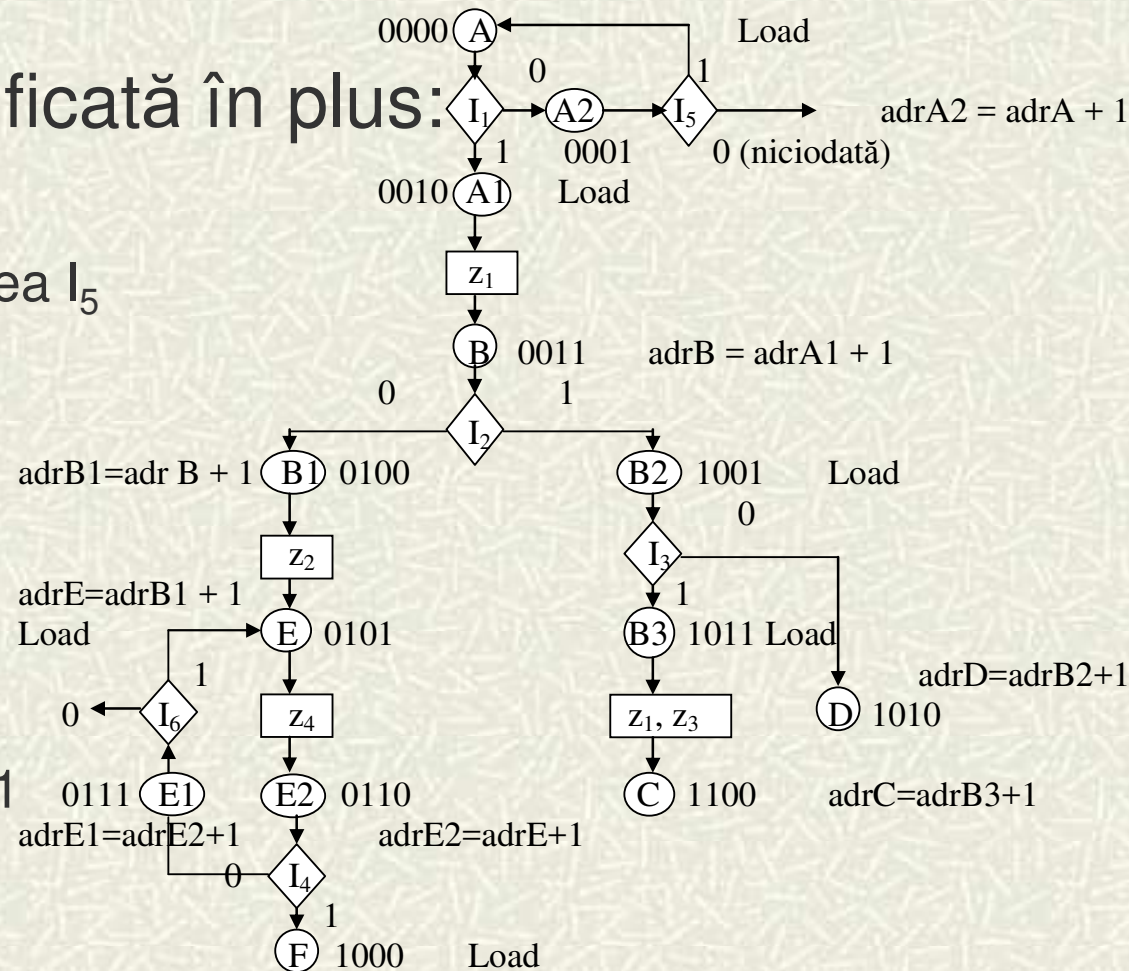


$\text{adrC} = \text{adrA} + 1$
format 1

2. ADRESARE PE STARE

Exemplu

- Organigrama modificată în plus:
 - după starea A_2
 - se introduce intrarea I_5
 - după starea E_1
 - se introduce I_6
 - după ieșirea z_4
 - se introduce E_2
 - intrările I_5 și I_6
 - iau doar valoarea 1



2. ADRESARE PE STARE

c. Adresare pe stare cu format variabil

- Capacitatea memoriei: $C = 2^m \times (s+m+1) = 128$ biți
- Codificarea intrărilor – pe 3 biți (3 linii de Sel la MUX)

	$S_2 S_1 S_0$
I_1	000
I_2	001
I_3	010
I_4	011
I_5	100
I_6	101

2. ADRESARE PE STARE

c. Adresare pe stare cu format variabil

■ Harta memoriei

Stare	Adresă	Format	Sel	S _{ua}
			Z ₄ Z ₃ Z ₂ Z ₁	
A	0 (0000)	1	000 (I ₁)	0010
A2	1 (0001)	1	100 (I ₅)	0000
A1	2 (0010)	0	0000001	
B	3 (0011)	1	001 (I ₂)	1001
B1	4 (0100)	0	0000010	
E	5 (0101)	0	0001000	
E2	6 (0110)	1	011 (I ₄)	1000
E1	7 (0111)	1	101 (I ₆)	0101
F	8 (1000)	?	?	?
B2	9 (1001)	1	010 (I ₃)	1011
D	0A (1010)	?	?	?
B3	0B (1011)	0	0000101	
C	0C (1100)	?	?	?

- **Observație:** Numărul biților pentru Sel + S_{ua} = numărul biților pentru ieșiri